

KeenPay

Agentic commerce with bounded AI

KeenPay — API spec

API specification

01 September 2026

<https://github.com/ambitiouss22/KeenPay>

KeenPay — API spec

Contracts for REST, WebSocket, and Razorpay webhooks. Implement against this; change the spec before changing behavior.

- Production base: `https://api.keenpay.example`
- Local: `http://localhost:8000`
- Prefix: `/api/v1`
- WebSocket: `/ws/v1/session`
- JSON unless noted

Money endpoints never trust client-supplied totals — server uses `approved_offer.final_amount_paise` from guardrail state.

1. Authentication

1.1 REST

Authorization: Bearer <JWT>

JWT claims:

```
{
  "sub": "user_abc123",
  "merchant_id": "merchant_keen",
  "role": "shopper",
  "exp": 1735689600
}
```

JSON

1.2 WebSocket

`wss://api.keenpay.example/ws/v1/session?token=<JWT>&session_id=<uuid>`

- `session_id` optional on connect; server creates one if omitted.
- Invalid/expired token → close code `4001`.

2. REST Endpoints

2.1 Health

GET `/api/v1/health`

Response 200:

```
{
  "status": "ok",
  "degradation_level": 0,
  "components": {
    "postgresql": "up",
    "redis": "up",
    "razorpay": "up",
    "llm": "up"
  },
  "version": "1.0.0"
}
```

JSON

2.2 Sessions

POST /api/v1/sessions

Create a negotiation session.

Request:

```
{
  "merchant_id": "merchant_keen",
  "metadata": {
    "utm_source": "web"
  }
}
```

JSON

Response 201:

```
{
  "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "status": "active",
  "created_at": "2026-08-28T18:00:00Z",
  "ws_url": "/ws/v1/session?session_id=7c9e6679-7425-40de-944b-e07fc1f90ae7"
}
```

JSON

GET /api/v1/sessions/{session_id}

Response 200:

```
{
  "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "status": "awaiting_confirmation",
  "negotiation_round": 2,
  "proposed_offer": null,
  "approved_offer": {
    "version": 2,
    "line_items": [
      {
        "sku": "HOODIE-NAVY-M",
        "product_id": "prod_001",
        "name": "Keen Hoodie Navy M",
        "quantity": 2,
        "list_unit_price_paise": 249900,
        "negotiated_unit_price_paise": 224900
      }
    ],
    "discount_pct": 10.0,
    "discount_amount_paise": 50000,
    "subtotal_paise": 499800,
    "final_amount_paise": 449800,
    "currency": "INR",
    "rationale": "10% loyalty discount applied"
  },
  "guardrail_decision": "APPROVED",
  "guardrail_decision_id": "dec_550e8400",
  "order_id": null,
  "payment_link_url": null
}
```

JSON

2.3 Catalog

GET /api/v1/catalog/products

Query params:

Param	Type	Description
q	string	Full-text search
sku	string	Exact SKU
limit	int	Default 20, max 50
offset	int	Pagination

Response 200:

```
{
  "items": [
    {
      "id": "prod_001",
      "sku": "HOODIE-NAVY-M",
      "name": "Keen Hoodie Navy M",
      "description": "Premium cotton hoodie",
      "list_price_paise": 249900,
      "cost_paise": 120000,
      "quantity_on_hand": 47,
      "quantity_available": 45,
      "attributes": { "color": "navy", "size": "M" },
      "active": true
    }
  ],
  "total": 1,
  "limit": 20,
  "offset": 0
}
```

JSON

GET /api/v1/catalog/products/{sku}

Response 200: Single product object (same shape as item above). Response 404: { "error": { "code": "PRODUCT_NOT_FOUND", "message": "..." } }

2.4 Chat (REST fallback)

POST /api/v1/sessions/{session_id}/messages

Synchronous chat turn (prefer WebSocket for production).

Request:

```
{
  "text": "I want 2 navy hoodies, medium, best price"
}
```

JSON

Response 200:

```
{
  "message_id": "msg_01j8x",
  "role": "assistant",
  "text": "I found the Keen Hoodie Navy M at ₹2,499 each. I can offer 10% off—₹4,498 total for 2. Shall I create a payment link?",
  "structured": {
    "type": "offer_summary",
    "approved_offer": { "...": "ProposedOffer shape" },
    "awaiting_confirmation": true
  },
  "trace_event_ids": ["evt_001", "evt_002"]
}
```

JSON

2.5 Confirm Payment

POST /api/v1/sessions/{session_id}/confirm

Request:

```
{
  "confirmed": true,
  "idempotency_key": "confirm-7c9e6679-v2"
}
```

JSON

Response 200:

```
{
  "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "order_id": "ord_9f3a2b1c",
  "payment_link_id": "plink_MNopQrStUv",
  "payment_link_url": "https://rzp.io/i/abc123",
  "final_amount_paise": 449800,
  "currency": "INR",
  "expires_at": "2026-08-29T18:00:00Z"
}
```

JSON

Response 409 (guardrail not approved):

```
{
  "error": {
    "code": "GUARDRAIL_NOT_APPROVED",
    "message": "Offer must pass guardrails before payment",
    "decision_id": null
  }
}
```

JSON

2.6 Orders

GET /api/v1/orders/{order_id}**Response 200:**

```
{
  "id": "ord_9f3a2b1c",
  "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "status": "pending",
  "final_amount_paise": 449800,
  "currency": "INR",
  "razorpay_payment_link_id": "plink_MNopQrStUv",
  "razorpay_payment_id": null,
  "line_items": [],
  "guardrail_decision_id": "dec_550e8400",
  "offer_version": 2,
  "created_at": "2026-08-28T18:05:00Z",
  "paid_at": null
}
```

JSON

2.7 Audit (read-only)

GET /api/v1/sessions/{session_id}/audit

Query: `limit` (default 50), `offset`

Response 200:

```
{
  "items": [
    {
      "id": "aud_001",
      "actor": "policy_engine",
      "action": "GUARDRAIL_EVALUATED",
      "decision_id": "dec_550e8400",
      "offer_version": 2,
      "input_snapshot": {},
      "output_snapshot": {},
      "created_at": "2026-08-28T18:04:30Z"
    }
  ],
  "total": 12
}
```

JSON

2.8 Admin — Escalations

GET /api/v1/admin/escalations

Requires `role: support_agent | manager`.

POST /api/v1/admin/escalations/{ticket_id}/resolve

Request:

```
{
  "resolution": "approve_override",
  "override_discount_pct": 12.0,
  "note": "Loyalty customer"
}
```

JSON

2.9 Razorpay Webhook

POST /webhooks/razorpay

Headers:

Header	Required
X-Razorpay-Signature	Yes
X-Razorpay-Event-Id	Yes

Raw body: Razorpay event JSON (signature computed on raw bytes).

Response 200: { "received": true } Response 400: Invalid signature Response 409: Amount mismatch → logged as disputed

3. WebSocket Protocol

3.1 Envelope

All messages use JSON envelope:

```
{
  "type": "<message_type>",
  "request_id": "req_optional_uuid",
  "timestamp": "2026-08-28T18:00:00.000Z",
  "payload": {}
}
```

JSON

3.2 Client → Server

`chat.message`

```
{
  "type": "chat.message",
  "request_id": "req_001",
  "payload": {
    "text": "Can you do 20% off on 2 hoodies?"
  }
}
```

JSON

`chat.confirm_payment`

```
{
  "type": "chat.confirm_payment",
  "request_id": "req_002",
  "payload": {
    "confirmed": true,
    "idempotency_key": "confirm-7c9e6679-v2"
  }
}
```

JSON

`trace.subscribe`

```
{
  "type": "trace.subscribe",
  "payload": {
    "include_payload": true,
    "filter": ["guardrail.*", "graph.*"]
  }
}
```

JSON

`ping`


```
{ "type": "ping", "payload": {} }
```

JSON

3.3 Server → Client

chat.response

```
{
  "type": "chat.response",
  "request_id": "req_001",
  "payload": {
    "message_id": "msg_01j8x",
    "role": "assistant",
    "text": "The best authorized price is ₹4,498 (10% off).",
    "structured": {
      "type": "offer_summary",
      "approved_offer": { "...": "ProposedOffer" },
      "awaiting_confirmation": true
    }
  }
}
```

JSON

trace.event

```
{
  "type": "trace.event",
  "payload": {
    "event_id": "evt_003",
    "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "timestamp": "2026-08-28T18:04:30.123Z",
    "event_type": "guardrail.rule.eval",
    "node_name": "guardrail_check",
    "duration_ms": 4,
    "payload": {
      "rule_id": "RULE_MAX_DISCOUNT",
      "passed": false,
      "action": "CLAMP",
      "message": "Discount capped at 15%",
      "inputs": { "requested_pct": 20, "max_pct": 15 }
    }
  }
}
```

JSON

order.status_updated

```
{
  "type": "order.status_updated",
  "payload": {
    "order_id": "ord_9f3a2b1c",
    "status": "paid",
    "razorpay_payment_id": "pay_KlMnOpQr",
    "paid_at": "2026-08-28T18:10:00Z"
  }
}
```

JSON

payment.link_ready

```
{
  "type": "payment.link_ready",
  "payload": {
    "order_id": "ord_9f3a2b1c",
    "payment_link_url": "https://rzp.io/i/abc123",
    "final_amount_paise": 449800,
    "expires_at": "2026-08-29T18:05:00Z"
  }
}
```

JSON

error

```
{
  "type": "error",
  "request_id": "req_001",
  "payload": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many messages. Retry in 30 seconds.",
    "retry_after_seconds": 30
  }
}
```

JSON

pong

```
{ "type": "pong", "payload": { "server_time": "2026-08-28T18:00:01Z" } }
```

JSON

3.4 Connection Lifecycle

Event	Code	Reason
Auth failure	4001	unauthorized
Session not found	4004	session_not_found
Server shutdown	1012	service_restart

Heartbeat: Client `ping` every 30s; server `pong`. Missing 3 pongs → client reconnect with backoff.

4. Pydantic Models

Module: `api/schemas/`

4.1 Shared

```
from datetime import datetime
from typing import Any, Literal, Optional
from pydantic import BaseModel, Field, ConfigDict

class ErrorDetail(BaseModel):
    code: str
    message: str
    details: Optional[dict[str, Any]] = None

class ErrorResponse(BaseModel):
    error: ErrorDetail
```

Python

4.2 Catalog

```
class ProductAttributes(BaseModel):
    model_config = ConfigDict(extra="allow")
    color: Optional[str] = None
    size: Optional[str] = None

class ProductOut(BaseModel):
    id: str
    sku: str
    name: str
    description: Optional[str] = None
    list_price_paise: int = Field(..., ge=0)
    cost_paise: int = Field(..., ge=0)
    quantity_on_hand: int = Field(..., ge=0)
    quantity_available: int = Field(..., ge=0)
    attributes: ProductAttributes = Field(default_factory=ProductAttributes)
    active: bool = True

class ProductListResponse(BaseModel):
    items: list[ProductOut]
    total: int
    limit: int
    offset: int
```

Python

4.3 Offers & Line Items

```

class LineItemOut(BaseModel):
    sku: str
    product_id: str
    name: str
    quantity: int = Field(..., ge=1)
    list_unit_price_paise: int = Field(..., ge=0)
    negotiated_unit_price_paise: Optional[int] = Field(None, ge=0)

class ProposedOfferOut(BaseModel):
    version: int = Field(..., ge=1)
    line_items: list[LineItemOut]
    discount_pct: float = Field(..., ge=0, le=100)
    discount_amount_paise: int = Field(..., ge=0)
    subtotal_paise: int = Field(..., ge=0)
    final_amount_paise: int = Field(..., ge=0)
    currency: Literal["INR"] = "INR"
    rationale: str

```

Python

4.4 Sessions

```

class SessionCreateRequest(BaseModel):
    merchant_id: str = "merchant_keen"
    metadata: dict[str, Any] = Field(default_factory=dict)

class SessionCreateResponse(BaseModel):
    session_id: str
    status: Literal["active", "closed"]
    created_at: datetime
    ws_url: str

class SessionStatusResponse(BaseModel):
    session_id: str
    status: Literal[
        "active",
        "negotiating",
        "awaiting_confirmation",
        "payment_pending",
        "paid",
        "escalated",
        "closed",
    ]
    negotiation_round: int
    proposed_offer: Optional[ProposedOfferOut] = None
    approved_offer: Optional[ProposedOfferOut] = None
    guardrail_decision: Optional[Literal["APPROVED", "REJECTED", "ESCALATED"]] = None
    guardrail_decision_id: Optional[str] = None
    order_id: Optional[str] = None
    payment_link_url: Optional[str] = None

```

Python

4.5 Chat

```

class ChatMessageRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=4000)

class StructuredChatPayload(BaseModel):
    type: Literal["offer_summary", "product_list", "clarification", "error"]
    approved_offer: Optional[ProposedOfferOut] = None
    products: Optional[list[ProductOut]] = None
    awaiting_confirmation: bool = False

class ChatMessageResponse(BaseModel):
    message_id: str
    role: Literal["assistant", "system"]
    text: str
    structured: Optional[StructuredChatPayload] = None
    trace_event_ids: list[str] = Field(default_factory=list)

```

Python

4.6 Payment Confirmation

```

class PaymentConfirmRequest(BaseModel):
    confirmed: bool
    idempotency_key: str = Field(..., min_length=8, max_length=128)

class PaymentConfirmResponse(BaseModel):
    session_id: str
    order_id: str
    payment_link_id: str
    payment_link_url: str
    final_amount_paise: int
    currency: Literal["INR"] = "INR"
    expires_at: datetime

```

Python

4.7 Orders

```

class OrderLineItemOut(BaseModel):
    sku: str
    name: str
    quantity: int
    unit_price_paise: int
    line_total_paise: int

class OrderOut(BaseModel):
    id: str
    session_id: str
    status: Literal["pending", "paid", "expired", "cancelled", "payment_disputed"]
    final_amount_paise: int
    currency: Literal["INR"] = "INR"
    razorpay_payment_link_id: Optional[str] = None
    razorpay_payment_id: Optional[str] = None
    line_items: list[OrderLineItemOut]
    guardrail_decision_id: str
    offer_version: int
    created_at: datetime
    paid_at: Optional[datetime] = None

```

Python

4.8 Audit

```
class AuditLogOut(BaseModel):
    id: str
    actor: Literal["agent", "policy_engine", "user", "system", "webhook", "human"]
    action: str
    decision_id: Optional[str] = None
    offer_version: Optional[int] = None
    input_snapshot: dict[str, Any]
    output_snapshot: dict[str, Any]
    created_at: datetime

class AuditLogListResponse(BaseModel):
    items: list[AuditLogOut]
    total: int
```

Python

4.9 WebSocket Envelope

```
from typing import Generic, TypeVar

T = TypeVar("T")

class WSEnvelope(BaseModel, Generic[T]):
    type: str
    request_id: Optional[str] = None
    timestamp: datetime
    payload: T

class WSChatMessagePayload(BaseModel):
    text: str

class WSChatConfirmPayload(BaseModel):
    confirmed: bool
    idempotency_key: str

class WSTraceEventPayload(BaseModel):
    event_id: str
    session_id: str
    timestamp: datetime
    event_type: str
    node_name: Optional[str] = None
    duration_ms: Optional[int] = None
    payload: dict[str, Any]
```

Python

4.10 Guardrail (internal API)

```

class RuleResultOut(BaseModel):
    rule_id: str
    passed: bool
    action: Literal["PASS", "CLAMP", "REJECT", "ESCALATE"] = "PASS"
    message: Optional[str] = None
    metadata: dict[str, Any] = Field(default_factory=dict)

class GuardrailDecisionOut(BaseModel):
    decision_id: str
    outcome: Literal["APPROVED", "REJECTED", "ESCALATED"]
    rule_results: list[RuleResultOut]
    evaluated_at: datetime
    policy_version: str
    approved_offer: Optional[ProposedOfferOut] = None

```

Python

5. Razorpay integration (test + production)

Use Razorpay test keys locally. Link creation runs only after `assert_payment_gates()` in `api/services/razorpay.py`. The amount sent to Razorpay always comes from `orders.final_amount_paise` on the server — never from the client payload.

5.1 Configuration

```

RAZORPAY_BASE_URL = "https://api.razorpay.com/v1"
# Sandbox test keys from Razorpay Dashboard
RAZORPAY_KEY_ID = "rzp_test_xxxx"
RAZORPAY_KEY_SECRET = "xxxx"
RAZORPAY_WEBHOOK_SECRET = "whsec_xxxx"

```

Python

5.2 Create Payment Link

Internal service: `RazorpayClient.create_payment_link(order: OrderOut) -> PaymentLinkResult`

HTTP Request:

```

POST /v1/payment_links HTTP/1.1
Host: api.razorpay.com
Authorization: Basic base64(key_id:key_secret)
Content-Type: application/json
Idempotency-Key: keenpay-{session_id}-v{offer_version}

{
  "amount": 449800,
  "currency": "INR",
  "accept_partial": false,
  "description": "KeenPay Order ord_9f3a2b1c",
  "reference_id": "ord_9f3a2b1c",
  "customer": {
    "name": "Priya Sharma",
    "email": "priya@example.com",
    "contact": "+919876543210"
  },
  "notify": {
    "sms": false,
    "email": false
  },
  "reminder_enable": false,
  "notes": {
    "session_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "guardrail_decision_id": "dec_550e8400",
    "offer_version": "2"
  },
  "callback_url": "https://app.keenpay.example/checkout/callback",
  "callback_method": "get",
  "expire_by": 1756512300
}

```

HTTP

Response 200:

```

{
  "id": "plink_MNopQrStUv",
  "short_url": "https://rzp.io/i/abc123",
  "amount": 449800,
  "currency": "INR",
  "status": "created",
  "reference_id": "ord_9f3a2b1c",
  "created_at": 1756425900
}

```

JSON

5.3 Mock Razorpay Client (Local / Tests)

Module: `api/services/razorpay_mock.py`

```

class MockRazorpayClient:
    async def create_payment_link(self, payload: dict, idempotency_key: str) -> dict:
        # Returns same shape as production; stores in memory dict
        ...

    async def simulate_payment(self, payment_link_id: str) -> None:
        # POST to local webhook injector for staging tests
        ...

```

Python

Mock endpoint (dev only):

POST /api/v1/dev/razorpay/simulate-payment

```
{
  "payment_link_id": "plink_MNopQrStUv",
  "payment_id": "pay_test_001"
}
```

JSON

Triggers internal webhook dispatch with valid test signature.

5.4 Webhook Events Handled

Event	Action
payment_link.paid	Primary — mark order paid
payment.captured	Fallback if payment entity linked
payment_link.expired	Mark order expired , release inventory
payment.failed	Log; keep order pending

Example **payment_link.paid** payload:

```
{
  "entity": "event",
  "account_id": "acc_xxx",
  "event": "payment_link.paid",
  "contains": ["payment_link", "payment"],
  "payload": {
    "payment_link": {
      "entity": {
        "id": "plink_MNopQrStUv",
        "amount": 449800,
        "currency": "INR",
        "status": "paid",
        "reference_id": "ord_9f3a2b1c"
      }
    },
    "payment": {
      "entity": {
        "id": "pay_KlMnOpQr",
        "amount": 449800,
        "currency": "INR",
        "status": "captured"
      }
    }
  },
  "created_at": 1756426000
}
```

JSON

5.5 Signature Verification

```
import hmac
import hashlib

def verify_razorpay_signature(body: bytes, signature: str, secret: str) -> bool:
    expected = hmac.new(
        secret.encode("utf-8"),
        body,
        hashlib.sha256,
    ).hexdigest()
    return hmac.compare_digest(expected, signature)
```

Python

6. Error Codes

Code	HTTP	Description
UNAUTHORIZED	401	Invalid JWT
SESSION_NOT_FOUND	404	Unknown session
PRODUCT_NOT_FOUND	404	Unknown SKU
GUARDRAIL_NOT_APPROVED	409	Payment blocked
CONFIRMATION_REQUIRED	409	User has not confirmed
INVENTORY_UNAVAILABLE	409	Stock insufficient
RATE_LIMIT_EXCEEDED	429	Too many requests
PAYMENT_LINK_FAILED	502	Razorpay error
WEBHOOK_SIGNATURE_INVALID	400	Bad signature
WEBHOOK_AMOUNT_MISMATCH	409	Disputed payment
INTERNAL_ERROR	500	Unexpected

7. Idempotency

Operation	Key Format
Payment confirm	Client-provided <code>idempotency_key</code>
Payment link	<code>keenpay-{session_id}-v{offer_version}</code>
Webhook	<code>X-Razorpay-Event-Id</code> stored UNIQUE

Duplicate idempotent requests return same response body with `200/201`.

8. Rate Limits (HTTP)

Endpoint	Limit
POST .../messages	30/min/user
POST .../confirm	3/hour/session
GET /catalog/*	120/min/IP
Webhook	100/min/IP

Headers on 429:

```
Retry-After: 30
X-RateLimit-Remaining: 0
```

9. OpenAPI

FastAPI auto-generates OpenAPI 3.1 at:

```
GET /openapi.json
GET /docs          # Swagger UI
GET /redoc         # ReDoc
```

Tags: `health`, `sessions`, `catalog`, `chat`, `orders`, `audit`, `admin`, `webhooks`.